# Evidence Frontiers

Physics-Guided Speculative Sandboxes for Agentic Development

Yossi Eliaz

Cambridge SRG seminar research companion | 13 October 2026

Cambridge repository | Evidence-aware reduction

## Abstract

Forkable execution makes it possible to compute futures before their premises have been verified. The systems problem is not creating more branches: it is choosing which dependencies to execute ahead, which hypotheses to distinguish, and when their results merit promotion. We propose *evidence frontiers*: an executable DAG may advance speculatively while a controller-owned frontier advances only across trustworthy, artifact-bound checks. We connect the scheduler to expected accepted prefixes in speculative decoding, committor probabilities in transition-path theory, value of information, rare-event splitting, and common-random-number experimental design. The key distinction is that correlation can *help* causal comparisons yet *hurt* independent-evidence claims. We derive simple economic and statistical decision rules, specify an end-to-end protocol and falsifiable benchmarks, and audit the mathematics with a dependency-free script. This is a research design, not a measured integrated sandbox runtime.

## 1 The problem: optimize the critical path, not the fork count

The Cambridge factory case shows the danger of optimizing an attractive but irrelevant number. One measured serial repair run spent 20.3 s in initial setup and 2,557.517 s of recorded stage time: setup was only 0.794%. An imagined 100× setup speedup saves about 20.1 s, not an order of magnitude overall. Its parallel edit-node critical path was a *calculation*, not an observed fork run; warm worktrees can also exploit file-level parallelism [1]. The runtime should compete against a *warm* alternative using time and money *to a verified result*, not create latency in isolation.

**Proposed abstraction: two frontiers.** A private *execution frontier* may run downstream of a provisional patch. A *trusted evidence frontier* only crosses a transition after an external checker validates its declared obligations on the exact artifact. A child may draft, but cannot edit the oracle, sign its own receipt, inherit publication authority, or unsend an external effect. This is CPU reorder-buffer thinking applied to agentic work, but not a first claim to speculation: Sherlock already overlaps downstream execution with verification [6]. We ask *where to fork, what to learn, and what the results prove.*

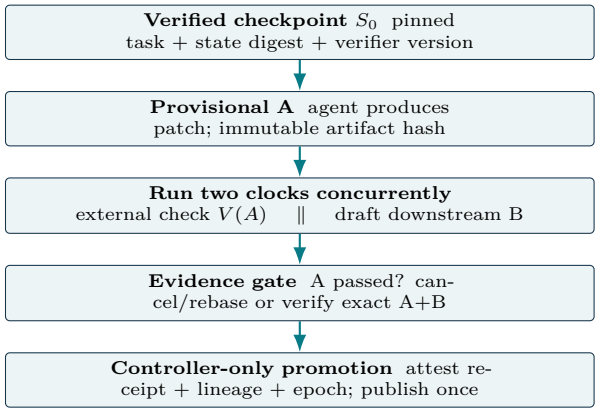



Figure 1: Execution can outrun evidence; publication cannot. External tool effects must be fenced *before* dispatch.

| Mode | Optimization target and hazard |
|---|---|
| **Run ahead** | Hide verification latency by executing a dependent continuation. A rejected ancestor invalidates descendants. |
| **Portfolio** | Find at least one verified solution. Overlapping failure modes make replicas redundant. |
| **Paired trial** | Compare interventions under matched conditions; shared environmental noise can *improve* precision. |

Table 1: A common snapshot does not imply the same objective or evidence rule.

## 2 Which kind of speculation? Three different jobs

These are not equivalent. A portfolio has an *OR* objective across alternative final artifacts. Run-ahead has an *AND* dependency along one executable path. Paired trials estimate a *difference*. Choosing the wrong mathematics for a mode leads either to wasted compute or false confidence.

## 3 What transfers from speculative decoding?

For target token law $p$ and cheap proposal $q$, exact speculative sampling accepts $x \sim q$ with $a(x) = \min\{1, p(x)/q(x)\}$ for $q(x) > 0$. Averaging gives

$$\alpha_{\text{tok}} = \sum_x \min\{p(x), q(x)\} = 1 - \text{TV}(p, q). \qquad (1)$$

A residual draw from $[p - q]_+ / \text{TV}(p, q)$ upon rejection restores the exact target distribution [3]. In contrast,

passing a software checker has no such correction theorem; even an independently controlled test suite may miss a wrong patch. EAGLE-2 and OPT-Tree motivate *context-dependent proposal trees*, not distribution-preservation guarantees for agent code [4, 5].

For $D$ dependent speculative transitions, let $L$ be the number of consecutive successfully checked transitions. For the true joint verifier events $A_j$,

$$\mathbb{E}[L] = \sum_{j=1}^{D} \mathbb{P}(A_1 \cap \cdots \cap A_j). \tag{2}$$

This tail-sum identity is exact; it does not assume independent checks. Yet equal-length prefixes need not have equal value: one transition may save an hour, another milliseconds. We should maximize *verified critical-path progress per unit cost*, not accepted-step count.

### 3.1 A Jeff-Dean-style crossover calculation

Suppose draft A takes $d_A$, checking A takes $v$, and follow-up draft B takes $d$; final checking costs $v_f$. If A passes and the resources do not contend:

$$T_{\text{serial}} = d_A + v + d + v_f, \tag{3}$$
$$T_{\text{ahead}} = d_A + \max\{v, d\} + v_f, \tag{4}$$
$$\Delta T_{\text{pass}} = \min\{v, d\}. \tag{5}$$

Let $a = \mathbb{P}(A \text{ passes})$, $h$ be added wall-time overhead, $c_B$ the cost of speculatively drafting B, $c_f$ extra fork/coordination compute, $\beta$ the value of a second saved and $\lambda$ the price of compute. Under this deliberately idealized two-path model—no contention, no latency penalty if A fails, and no reusable B after failure—run-ahead is worthwhile if

$$\boxed{\beta\, a \min\{v, d\} > \beta h + \lambda[(1-a)c_B + c_f].} \tag{6}$$

The left side is expected hidden waiting; the right charges overhead and wasted speculation. Example: $d_A = 10, v = 30, d = 20, v_f = 30$ seconds gives $90 \to 70$ seconds *conditional on A passing*; with $a = .8$, the ideal expected gross saving is 16 seconds, and the wasted B work is 4 worker-seconds when $c_B = 20$ worker-seconds. These are illustrations, *not* measurements.

**Queueing changes the answer.** A verifier with mean service demand $\mathbb{E}[V]$ and long-run arrival rate $\lambda_v$ has utilization $\rho_v = \lambda_v \mathbb{E}[V]$ for one equivalent server. A stable stationary queue requires $\rho_v < 1$; launching more branches can saturate the *verifier*, making both cost and latency worse. The scheduler needs the verifier's shadow price and tail latency, not just its throughput [7].

## 4 Physics suggests predicting destinations, not next checks

### 4.1 The committor: eventual verified success

For a complete Markov state $s$ under a *fixed* continuation policy $\pi$, define the committor

$$h_\pi(s) = \mathbb{P}_\pi(\tau_G < \tau_F \mid S_0 = s), \tag{7}$$

where $G$ is verified goal completion and $F$ is terminal failure. With transition kernel $P_\pi(s'|s)$,

$$h_\pi(s) = \sum_{s'} P_\pi(s'|s) h_\pi(s'), \quad h_\pi|_G = 1, \quad h_\pi|_F = 0. \tag{8}$$

This is the committor equation from transition-path theory [8]. A branch can fail its next superficial check yet be useful because it discovers a reproducer or root cause; conversely, a patch can pass easy checks while having little chance of eventual success. An *estimated* committor can rank states by expected future value, but it is policy-specific and must be validated out of sample.

There is a mathematically exact analogy worth keeping *separate* from software correctness: if $h$ is the true positive harmonic committor for a Markov process, the Doob transform

$$P^h(s'|s) = P(s'|s)\frac{h(s')}{h(s)} \tag{9}$$

constructs the process conditioned on reaching $G$ before $F$ (on appropriate nonabsorbing states). This is *not* an executable correctness guarantee for agents with unknown transitions; replacing $h$ by a learned score gives a *heuristic proposal policy*, not a valid exact conditioning theorem.

### 4.2 Rare-event splitting: fork where progress persists

Suppose success requires nested milestones $E_m \subseteq \cdots \subseteq E_1$, with positive conditioning probabilities. Then

$$\mathbb{P}(E_m) = \prod_{k=1}^{m} \mathbb{P}(E_k \mid E_{k-1}), \quad E_0 = \Omega. \tag{10}$$

Adaptive multilevel splitting retains trajectories reaching promising levels and resamples continuations instead of repeatedly paying for the entire prefix [9]. In coding, plausible *candidate milestones* are reproducer, root cause, targeted behavioral fix, integration check and final review. But a checkpoint is reusable only when its state is valid for the next intervention. Blindly cloning a flawed hypothesis creates genealogical collapse, not progress.

**Discovery is not probability estimation.** Retaining high-score trajectories changes the sampling law from $P$ to $Q$. To estimate an event's *original* probability under a valid importance-sampling setup requires

$$\mathbb{P}_P(G) = \mathbb{E}_Q\left[\frac{dP}{dQ}\,\mathbf{1}_G\right], \quad P \ll Q. \tag{11}$$

Without a justified likelihood ratio or a valid rare-event estimator, optimized branch populations cannot claim calibrated original frequencies. The same caution applies to evidence-aware statistical reduction.

## 5 The most useful fork may be a diagnostic test

Probability of passing the *next* check ignores the information gained from a failure. Suppose two root causes have probability 1/2 each, each corresponding repair costs 10 units, and a perfect discriminating diagnostic costs 2. Blindly try repair A, then B if needed:

$$\mathbb{E}[C_{\text{repairs}}] = 10 + \tfrac{1}{2}(10) = 15.$$

Diagnose first and make the correct repair:

$$\mathbb{E}[C_{\text{diagnostic}}] = 2 + 10 = 12.$$

The test produces no patch yet saves three expected cost units. It has *value of information* (VOI).

With current information $H$, candidate terminal action $a$, experiment $e$ with result $O_e$, utility $U$, and experiment cost $C(e)$,

$$\begin{aligned}\mathrm{VOI}(e) = \mathbb{E}_{O_e|H}\Big[\max_a \mathbb{E}[U(a) \mid H, O_e]\Big] \\ - \max_a \mathbb{E}[U(a) \mid H] - C(e).\end{aligned} \tag{12}$$

The equation describes a single additional observation followed by one terminal decision; richer workflows need dynamic programming. The practical lesson is *schedule tests and interventions together*, not only proposals expected to pass. A deterministic test of the Cambridge `CellBusy`–same-work-order adoption path might be more valuable than another nearly identical repair attempt [1].

## 6 The correlation paradox: repeat, diversify, or pair?

### 6.1 Positive correlation can improve paired comparisons

Assume two matched experiments use the same environmental perturbation $U$:

$$Y_A = \theta_A + U + \varepsilon_A, \quad Y_B = \theta_B + U + \varepsilon_B. \tag{13}$$

For mutually independent noise terms with $\mathrm{Var}(U) = \tau^2$ and $\mathrm{Var}(\varepsilon_i) = \sigma^2$, shared-$U$ pairing yields

$$\boxed{\mathrm{Var}(Y_A - Y_B) = 2\sigma^2.} \tag{14}$$

Independent environmental draws instead yield $2(\tau^2 + \sigma^2)$. With $\tau^2 = 9, \sigma^2 = 1$, variance goes from 20 to 2—a 10× reduction *under this model* [10]. The general identity is $\mathrm{Var}(A - B) = \mathrm{Var}\, A + \mathrm{Var}\, B - 2\,\mathrm{Cov}(A, B)$: pairing helps only when the induced covariance is positive. Reuse identical *test fixtures and controlled input randomness*, never inherited secrets or clone identity/RNG for security.

### 6.2 But dependence reduces search coverage

For a fixed set $S$ of candidate acceptance events $A_i$,

$$\begin{aligned}F(S) &= \mathbb{P}\left(\bigcup_{i\in S} A_i\right), \\ \Delta_j(S) &= F(S \cup \{j\}) - F(S) \\ &= \mathbb{P}\left(A_j \cap \bigcap_{i\in S} A_i^c\right).\end{aligned} \tag{15}$$

This is monotone and submodular for a *fixed* joint law: new workers have diminishing marginal coverage as the set grows. It does not imply a greedy guarantee for a history-dependent controller. As a counterexample, if exactly one of two mutually exclusive approaches A/B succeeds, then $\mathbb{P}(B) = 1/2$ yet $\mathbb{P}(B \mid A^c) = 1$: failure *raises* the other branch's value. Estimate conditional probabilities from held-out family-aware observations, not independence shortcuts.

Finally, in correlated numerical estimation, $K$ equally precise workers with equicorrelation $\rho \geq 0$ have

$$K_{\text{eff}} = \frac{K}{1 + (K-1)\rho}. \tag{16}$$

That result concerns *estimation-error correlation*, not binary patch-success correlation. Shared evidence identifiers and fork lineage are not enough to identify covariance. The previously published reducer rejects declared duplicate IDs but does not fit this covariance model [2].

## 7 An implementable controller, not a physics metaphor

**State.** A trusted controller pins (`task`, `base`, `manifest`, `verifier`, `epoch`), maintains a directed acyclic graph of provisional artifact hashes, dependency receipts, rejected and censored outcomes, executor costs and effect logs. The policy state also contains a calibrated belief over root-cause hypotheses. Never interpret a timeout as a witnessed test failure.

**Safety gates.** Verify capture fidelity and source identity; private writable state; fresh capability/credential issuance; blocked or mediated network/tool effects; budget and resource limits. Copying guest memory does not copy authority. If fidelity or security fails, do not fork. A *warm worktree* must remain a first-class alternative.

**Choice.** The action set contains {diagnose, run ahead, diversify, paired compare, verify now, warm serial, stop}. For an idealized belief-state controller, the Bellman form is

$$\begin{aligned}V(b, s, B) = \max_{a \in \mathcal{A}_{\text{safe}}(s,B)} \Big\{ \\ r_a - \lambda c_a + \mathbb{E}_{o|b,s,a} V(b_o, s_o, B - k_a)\Big\},\end{aligned} \tag{17}$$

where $b_o, s_o$ are the updated belief and state, $c_a$ is economic cost and $k_a$ consumes budget $B$. Terminal rewards

are assigned to an *externally verified* exact artifact. This is a *formulation*, not a solved optimal runtime. A practical heuristic first ranks cheap discriminating tests by VOI, then ranks admissible forks by conditional *marginal* verified progress and potential critical-path savings, rejecting choices with negative net gain or saturated verifier queues. The committor is a learned feature, not an oracle.

**Commit.** A child returns bytes, digest and trace; an external, pinned checker issues a controller-authenticated receipt tying *actual* verifier identity to those bytes and to the epoch. Only a current-epoch, fully checked result can move the evidence frontier. Verify the *final composed artifact* after merging patches. Persist artifacts and ledger before teardown, use atomic compare-and-swap on promotion, and block observable tool calls before issue. A green test proves only its stated predicate; repeated search may exploit an incomplete oracle. For illustration, if each of 100 wrong proposals had an independent 1% false-pass chance, the chance of at least one false pass would be $1 - .99^{100} \approx 63.4\%$. Real false passes need not be independent; these figures are not a measured bug rate.

**Exploration temperature (optional).** For a fixed finite candidate set with prior proposal $q_0(a|s) > 0$, maximizing $\mathbb{E}_{a\sim\pi}Q(s,a) - TD_{\mathrm{KL}}(\pi\|q_0)$ yields

$$\pi_T(a|s) \propto q_0(a|s)\exp\{Q(s,a)/T\}. \tag{18}$$

Temperature can encourage hypothesis diversity; this is a scheduling *choice*, not an exact-decoding acceptance correction. Reserve exploration budget when the learned committor is uncertain or when repeated failures expose a common-mode hypothesis.

## 8 Evaluation: what would change our minds?

| Question | Required comparison / falsifier |
|---|---|
| **Better than warm?** | Warm serial, worktree portfolio, fixed-width forks, adaptive policy; same task, verifier and capped hardware. Reject advantage when warm cost/time dominates. |
| **Hide waiting?** | Two-stage A→B with A checked concurrently with drafting B. Measure verified critical path, wasted CPU/LLM time, p50/p95, rollback cost and contention. |
| **Learn from failure?** | Paired diagnostic-first versus direct repair; hold out repo/task families; assess Brier/log loss and decision utility. |
| **Control error?** | Controlled matched A/B forks versus independent environments; estimate paired covariance and confidence coverage across *independent* families. |
| **Trust gate?** | Challenge stale epochs, tampered checkers, duplicate evidence, unverified merged bytes and speculative observable tool calls. |

Table 2: Proposed experiments; none are claimed executed.

**Design discipline.** Preserve the Cambridge preregistered Gate 0, H1 warm-cache test, and H2 sibling-coupling test *unchanged* [1]. This note proposes a separate exploratory follow-up. Report all attempted tasks, partial failures, timeouts, rejected branches, policy propensities and held-out correctness checks. Failure to beat a warm baseline is a meaningful result. Use falsifying examples where more forks increase verifier load or where repeated testing finds a false pass. Exact success labels for semantic correctness require a genuinely stronger hold-out oracle, not the same suite repeatedly queried.

## Conclusion: a compiler for experiments

**Fork only when reached state is valid and reuse beats warm reconstruction.**
**Execute ahead to remove waiting; diversify to increase coverage; pair to isolate causal effects.**
**Spend the next unit of compute where its marginal verified progress *or information gain* is highest.**
**Promote exact bytes once; let only justified evidence increase confidence.**

## References


[1] Y. Eliaz, *Forkable Sandboxes: Cambridge Seminar*, 2026; `CLAIM-FENCE.md`, `PREREGISTRATION.md`.

[2] Y. Eliaz, *Evidence-Aware Reduction for Forkable Compute*, arXiv:2607.09689v4, 2026.

[3] Y. Leviathan, M. Kalman, Y. Matias, *Fast Inference from Transformers via Speculative Decoding*, ICML 2023.

[4] Y. Li et al., *EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees*, EMNLP 2024.

[5] J. Wang et al., *OPT-Tree: Speculative Decoding with Adaptive Draft Tree Structure*, TACL 2025.

[6] Y. Ro et al., *Sherlock: Reliable and Efficient Workflow Execution*, Microsoft Research, 2025.

[7] J. Dean, L. A. Barroso, *The Tail at Scale*, CACM 2013.

[8] W. E, E. Vanden-Eijnden, *Transition-Path Theory and Path-Finding Algorithms for the Study of Rare Events*, Annu. Rev. Phys. Chem. 2010.

[9] C.-E. Bréhier et al., *Unbiasedness of Some Generalized Adaptive Multilevel Splitting Algorithms*, Ann. Appl. Probab. 2016.

[10] J. P. C. Kleijnen, *Analyzing Simulation Experiments with Common Random Numbers*, Management Science, 1988.